State Merging in Symbolic Execution

MIR dumps, query relevance, and compact summaries

Yaroslav Riabtsev

Communication and Distributed Systems

https://comsys.rwth-aachen.de/

Aachen, July 2026

KLEE run vs. internal analysis

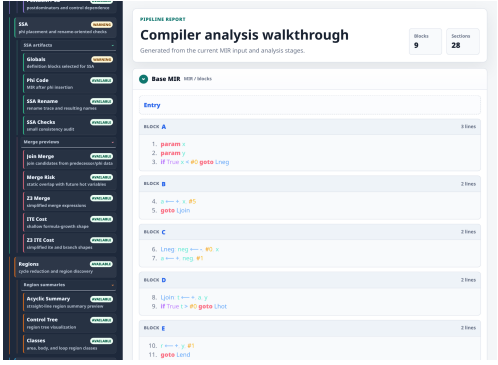
KLEE input program

```c
int count_matches_buggy(int i, int n, int target) {
    int matches = 0;

    while (i <= n) {
        if (target == i) {
            matches = matches + 1;
        } else {
            matches = matches + 0;
        }

        i = i + 1;
    }

    return matches;
}
```

Raw KLEE output excerpt

```
$ klee build/count_matches_bug.bc
KLEE: output directory is "/work/out/count_matches_bug"
KLEE: Using STP solver backend
...
KLEE: halting execution, dumping remaining states
KLEE: done: completed paths = 1
KLEE: done: partially completed paths = 6
KLEE: done: generated tests = 7
```

Internal analysis view



yaryabtsev.github.io/compilers-course/

Repeated symbolic branching

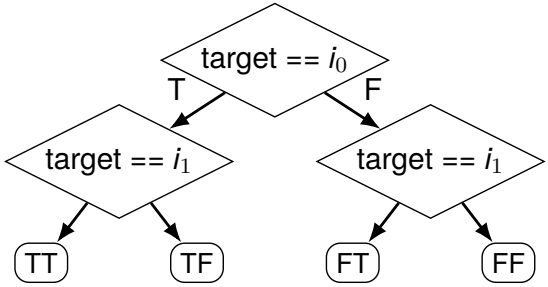
C++ source

```
int count_matches(int i, int n, int target) {
    int matches = 0;
    while (i < n) {
        if (target == i)
            matches = matches + 1;
        else
            matches = matches + 0;
        i = i + 1;
    }
    return matches;
}
```

MIR dump

```
matches <-- 0
Lloop:ifFalse i < n goto Lend
ifFalse target = i goto Lskip
matches <-- +, matches, 1
gotoLnext
Lskip: matches <-- +, matches, 0
Lnext:i <-- +, i, 1
gotoLloop
Lend: return matches
```

Path histories



$$1, 2, 4, 8, \ldots, 2^k$$

One branch and one join

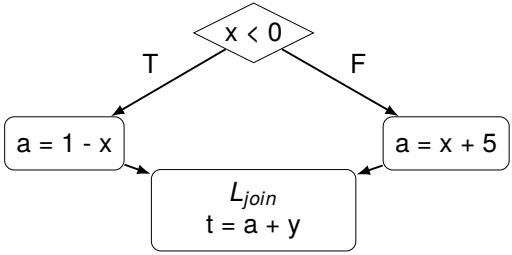
C++ fragment

```
int a;
if (x < 0) {
    a = 1 - x;
} else {
    a = x + 5;
}
int t = a + y;
```

MIR dump

```
ifTrue x < 0goto Lneg
a <-- +, x, 5
gotoLjoin
Lneg: neg <-- -, 0,x
a <-- +, neg, 1
Ljoin:t <-- +, a, y
```

CFG



Two states in the worklist

$$s = (\ell, \pi, \mu)$$

True branch: s_1

ℓ	L_{join}
π	$x < 0$
μ	$a \mapsto 1 - x$

False branch: s_2

ℓ	L_{join}
π	$\neg(x < 0)$
μ	$a \mapsto x + 5$

Worklist W

front $\longrightarrow$ $\boxed{s_1}$ $\boxed{s_2}$ $\boxed{\cdots}$

$(\ell, \pi, \mu) = pickNext(W)$

Merging the same two states

True branch: s_1

ℓ	L_{join}
π	$\pi_1 = x < 0$
μ	$a \mapsto 1 - x$

False branch: s_2

ℓ	L_{join}
π	$\pi_2 = \neg(x < 0)$
μ	$a \mapsto x + 5$

$\Downarrow$ merge-compatible: same location L_{join}

Merged state

ℓ	L_{join}
π	$\pi_1 \vee \pi_2$
μ	$a \mapsto \mathrm{ite}(\pi_1, 1 - x, x + 5)$

$$\pi = \pi_1 \vee \pi_2$$

$$\mu(a) = \mathrm{ite}(\pi_1, 1 - x, x + 5)$$

Sharing symbolic expressions

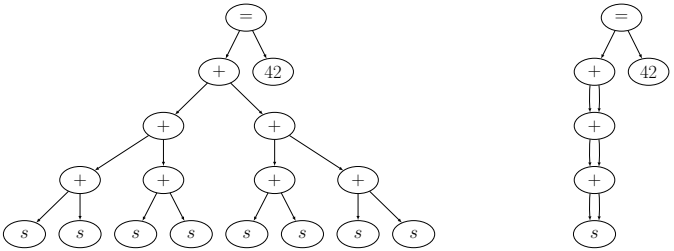


Figure 4: Hash consing example. Top-left: naively generated formula. Top-right: hash-consed formula.

Straight-line input

```
x = s; // s is symbolic
x = x + x;
x = x + x;
x = x + x;
assert(x == 42);
```

Avgerinos et al., Enhancing Symbolic Execution with Veritesting, ICSE 2014, Fig. 4.

Formula growth after the merge

Remaining code

```
int a;
if (x < 0) {
  a = 1 - x;
} else {
  a = x + 5;
}

int t = a + y;
int r;
if (t > 0)
  r = t + 1;
else
  r = y + 1;
```

Merged value

$$a = \text{ite}(x < 0,\ 1 - x,\ x + 5)$$

Substitute directly into *t*

$$t = \text{ite}(x < 0,\ 1 - x,\ x + 5) + y$$

Substitute directly into *r*

$$r = \text{ite}(\,\text{ite}(x < 0, 1 - x, x + 5) + y > 0,\ \text{ite}(x < 0, 1 - x, x + 5) + y + 1,\ y + 1)$$

Push additions into the *ite* branches

$$r = \text{ite}(\,\text{ite}(x < 0, 1 - x + y, x + 5 + y) > 0,\ \text{ite}(x < 0, 2 - x + y, x + 6 + y),\ y + 1)$$

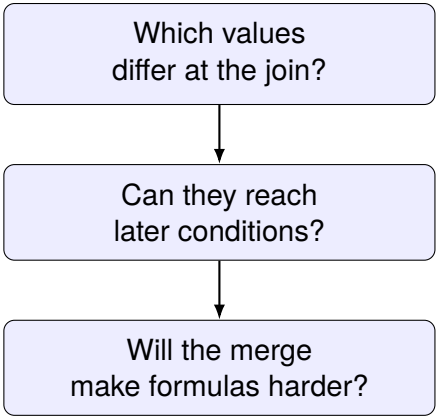
fewer states, repeated symbolic subexpressions

Before QCE: the merge question

Remaining code

```
int a;
if (x < 0) {
  a = 1 - x;
} else {
  a = x + 5;
}
// Ljoin: candidate merge of a
int t = a + y;
int r;
if (t > 0)
  r = t + 1;
else
  r = y + 1;
// Lend: candidate merge of r
```

a differs at $L_{join} \Rightarrow t = a + y \Rightarrow t > 0$



Candidate merge points

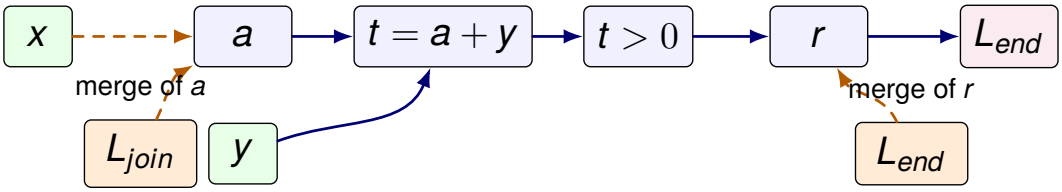
Candidate at L_{join}

```
int a;
if (x < 0) {
  a = 1 - x;
} else {
  a = x + 5;
}
// Ljoin: candidate merge of a
```

Candidate at L_{end}

```
int t = a + y;
int r;
if (t > 0) {
  r = t + 1;
} else {
  r = y + 1;
}
// Lend: candidate merge of r
```

Future dependency



What changes at each candidate?

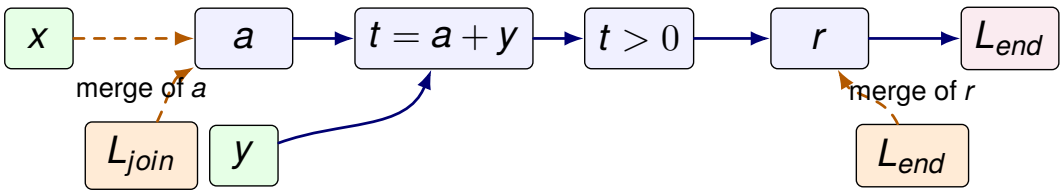
At L_{join}

differing source of a	$\{x\}$
future condition depends on	$\{x, y\}$
overlap	$\{x\}$

At L_{end}

r differs	yes
future condition	none
overlap	$\varnothing$

Future dependency



Project dump view

Merge-risk table

Join	Value	Diff sources	Future hot	Overlap	Result
L_{join}	a	$\{x\}$	$\{x, y\}$	$\{x\}$	mixed
L_{end}	r	$\{x, y\}$	$\varnothing$	$\varnothing$	no future query

Dependency recovery vs. manual QCE oracle

Analysis	Recovered for $t > 0$	What is added	Oracle L1
Manual QCE oracle	$\{x, y\}$	original C-relation + q-recurrence	reference
Syntactic	$\{t\}$	direct branch operands	10
Local	$\{a, y\}$	same-block def-use expansion	5
CFG	$\{x, y\}$	propagation through predecessors and joins	0
Versioned	$\{x_0, y_0\}$	definition-sensitive source tracking	0

Oracle L1 is computed in the source-variable domain; x_0, y_0 are normalized to x, y.

QCE shape from the paper

$$q(\ell, c) = \begin{cases} \beta q(succ(\ell), c) + \beta q(\ell', c) + c(\ell, e), & instr(\ell) = if(e)\ goto\ \ell' \\ 0, & instr(\ell) = halt \\ q(succ(\ell), c), & \text{otherwise} \end{cases}$$

$$Q_t(\ell) = q(\ell, c_t), \qquad Q_{add}(\ell, v) = q(\ell, c_{add,v})$$

$$Hot(\ell) = \{v \mid Q_{add}(\ell, v) > \alpha Q_t(\ell)\}$$

Parameters

$\alpha > 0$	threshold
$\beta \in (0.5, 1)$	attenuation
$\kappa \in \mathbb{N}$	preview bound

dump run: $\alpha = 0.35$, $\beta = 0.50$ boundary value

The formulas on the left are the paper-shaped reference.

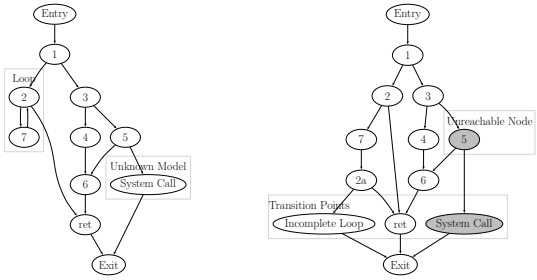
Kuznetsov et al., Efficient State Merging in Symbolic Execution, PLDI 2012.

Veritesting regions and transition points

Efficient merging
state-level merge and QCE

Veritesting
calls and returns often become transition points

Java Ranger
dynamically inline method regions



Veritesting on a program fragment with loops and system calls. (a) Recovered CFG. (b) CFG after transition point identification & loop unrolling. Unreachable nodes are shaded.

Avgerinos et al., Enhancing Symbolic Execution with Veritesting, ICSE 2014, Fig. 1.

Java call chain: raw bytecode view

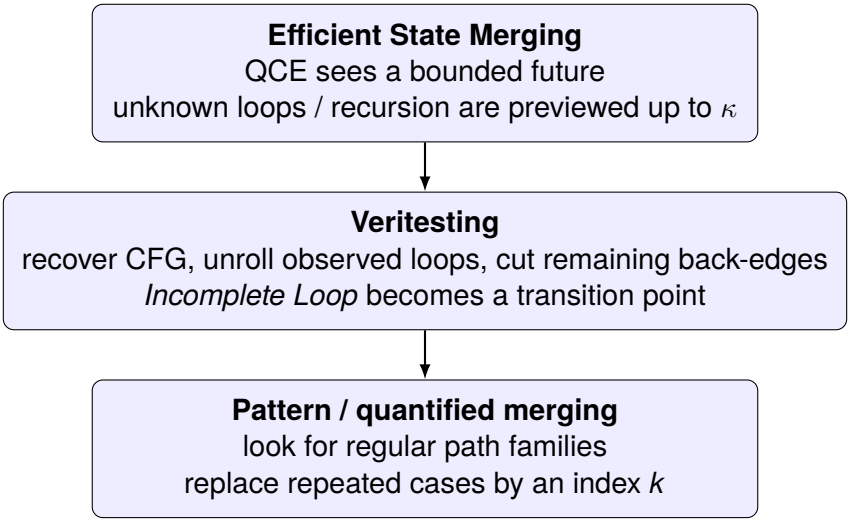
Efficient merging
state-level merge and QCE

Veritesting
calls and returns often become transition points

Java Ranger
dynamically inline method regions

```
// javap -c -p -s CallChainDemo

  public static int read(java.util.List<java.lang.Integer>, int);
    descriptor: (Ljava/util/List;I)I
    Code:
       0: aload_0
       1: iload_1
       2: invokeinterface #7,  2            // InterfaceMethod java/util/List.get:(I)Ljava/lang/Object;
       7: checkcast     #13                 // class java/lang/Integer
      10: invokevirtual #15                 // Method java/lang/Integer.intValue:()I
      13: ireturn


// javap -c -p -s java.util.ArrayList

  public E get(int);
    descriptor: (I)Ljava/lang/Object;
    Code:
       0: iload_1
       1: aload_0
       2: getfield      #48                 // Field size:I
       5: invokestatic  #117                // Method java/util/Objects.checkIndex:(II)I
       8: pop
       9: aload_0
      10: iload_1
      11: invokevirtual #122                // Method elementData:(I)Ljava/lang/Object;
      14: areturn


// javap -c -p -s java.lang.Integer

  public int intValue();
    descriptor: ()I
    Code:
       0: aload_0
       1: getfield      #204                // Field value:I
       4: ireturn
```

Loops across the papers

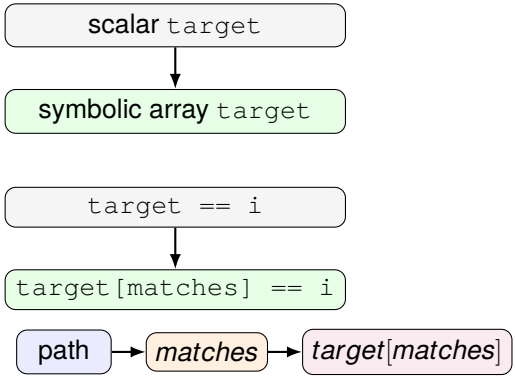


Why the two patches matter

Running example variant

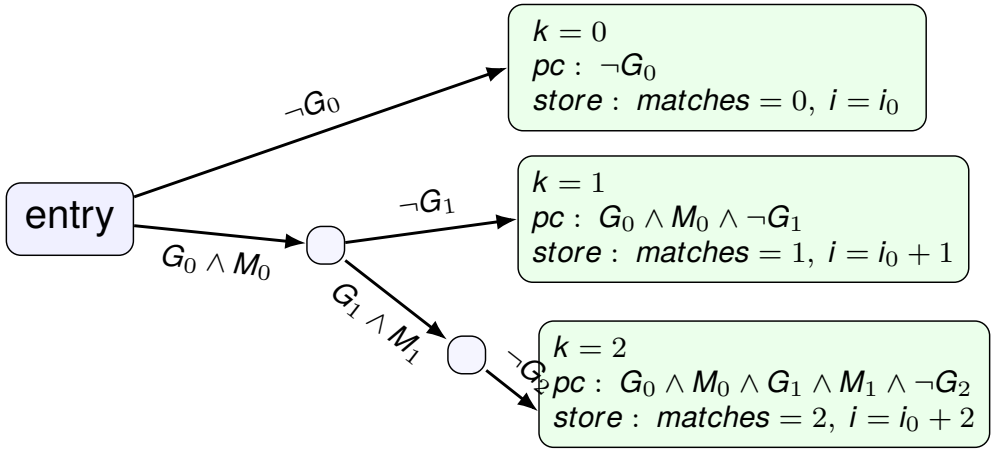
```
int count_matches(int i, int n, int* target) {
  int matches = 0;

  while (i < n) {
    if (target[matches] == i) {
      matches = matches + 1;
    } else {
      matches = matches + 0;
    }

    i = i + 1;
  }

  return matches;
}
```

Two conceptual patches



bounded n, symbolic array contents, sufficient target capacity

Which leaves can be merged together?

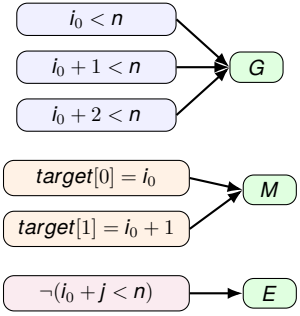


$G_j \equiv i_0 + j < n \quad M_j \equiv target[j] = i_0 + j$

Selected leaves: same exit, same store domain. Mismatch histories form other groups/fallback.

How the paper detects a regular partition

1. Hash conditions by structure



Numeric indices may vary; condition shape and branch polarity are preserved.

2. Encode root-to-leaf paths as words

$$k = 0 \quad : \quad W \cdot E$$
$$k = 1 \quad : \quad W \cdot GM \cdot E$$
$$k = 2 \quad : \quad W \cdot GM \cdot GM \cdot E$$

3. Detect shared regular form

$$h(\pi(n_k)) = \omega_1 \omega_2^k \omega_3$$

$\omega_1 = W$	common prefix
$\omega_2 = GM$	repeated successful iteration
$\omega_3 = E$	final exit

Regularity is detected in execution-tree constraints, not merely in the CFG loop.

Trabish et al., *State Merging with Quantifiers in Symbolic Execution.*

From path pattern to formula pattern

Observed instances

$k = 0 \quad : \quad \neg G_0$

$k = 1 \quad : \quad G_0 \wedge M_0 \wedge \neg G_1$

$k = 2 \quad : \quad G_0 \wedge M_0 \wedge G_1 \wedge M_1 \wedge \neg G_2$

Synthesized components

$$\varphi_1 = \mathit{true}$$

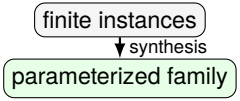
$$\varphi_2(j) = (i_0 + j < n) \wedge (\mathit{target}[j] = i_0 + j)$$

$$\varphi_3(k) = \neg(i_0 + k < n)$$

Common schema for each observed k

$$\mathit{tpc}(n_k) = \varphi_1 \wedge \bigwedge_{j=0}^{k-1} \varphi_2(j) \wedge \varphi_3(k)$$

$$k \in K \ \wedge \ \forall j.\, 0 \le j < k \Rightarrow \varphi_2(j) \ \wedge \ \varphi_3(k)$$



The paper synthesizes changing numeric terms, typically using an affine form $a \cdot k + b$.

What becomes smaller?

Standard merge

$$pc = pc_0 \vee pc_1 \vee pc_2$$

$$matches = ite(pc_0, 0,\ ite(pc_1, 1, 2))$$

$$i = ite(pc_0, i_0,\ ite(pc_1, i_0 + 1, i_0 + 2))$$

Pattern-based merge

$$k \in \{0, 1, 2\} \quad (0 \leq k \leq 2)$$

$$\forall j.\ 0 \leq j < k \Rightarrow (i_0 + j < n \wedge target[j] = i_0 + j)$$

$$\neg(i_0 + k < n)$$

$$matches = k, \qquad i = i_0 + k$$

k identifies the selected path case; the same k synthesizes the corresponding store values. The compact path constraint is equisatisfiable with the standard merge, and store values agree under the corresponding value of k.

If pattern synthesis fails, use standard merging. If one store value cannot be expressed as $t(k)$, retain its ordinary `ite`.

Trabish et al., *State Merging with Quantifiers in Symbolic Execution*.

Back to the project: MIR loop preview

Scalar MIR skeleton

```
matches <-- 0
Lloop:
  ifFalse i < n goto Lend
  ifFalse target == i goto Lskip
  matches <-- +, matches, 1
  goto Lnext
  Lskip: matches <-- +, matches, 0

Lnext:
  i <-- +, i, 1
  goto Lloop

Lend: return matches
```

The scalar loop is a structural preview, not a full quantified merge.

Structural roles recovered by the dump

MIR evidence	Structural role
`i < n`	loop guard
`target == i`	repeated symbolic branch
`matches + 1`	guarded accumulator
`i + 1`	induction update
`goto Lloop`	back edge

Candidate indexed family

$guard(k)$	$i_0 + k < n$
$branch(k)$	$target = i_0 + k$
$induction(k)$	$i = i_0 + k$
$accumulator$	$matches$

MIR structure $\rightarrow$ candidate repeated family $\rightarrow$ formula and store synthesis $\rightarrow$ quantified merged state

Beyond scalar MIR: where complexity moves

Harder to model

aliasing, bounds, object fields, heap updates

More structure to exploit

sharing, summaries, method regions, array patterns

What real engines carry

- KLEE / quantified: arrays, memory objects, solver terms
- Veritesting / Java Ranger: regions, calls, dispatch
- Efficient merging: store differences, hot variables, query cost

Limit of this MIR dump

Mostly scalar variables and CFG dependencies. Memory, aliasing, fields, and heap-shaped updates are only approximated here.

Next target: represent program structure better, not just shorten formulas.

Main sources

Papers

1. C. Cadar, D. Dunbar, and D. Engler. *KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*. OSDI, 2008.
2. V. Kuznetsov, J. Kinder, S. Bucur, and G. Candea. *Efficient State Merging in Symbolic Execution*. PLDI, 2012.
3. T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley. *Enhancing Symbolic Execution with Veritesting*. ICSE, 2014.
4. V. Sharma, S. Hussein, M. W. Whalen, S. McCamant, and W. Visser. *Java Ranger: Statically Summarizing Regions for Efficient Symbolic Execution of Java*. ESEC/FSE, 2020.
5. D. Trabish, N. Rinetzky, S. Shoham, and V. Sharma. *State Merging with Quantifiers in Symbolic Execution*. 2023.

Project artifacts

`github.com/yaryabtsev/compilers-course`

MIR dumps, CFGs, QCE tables, and merge-risk reports.